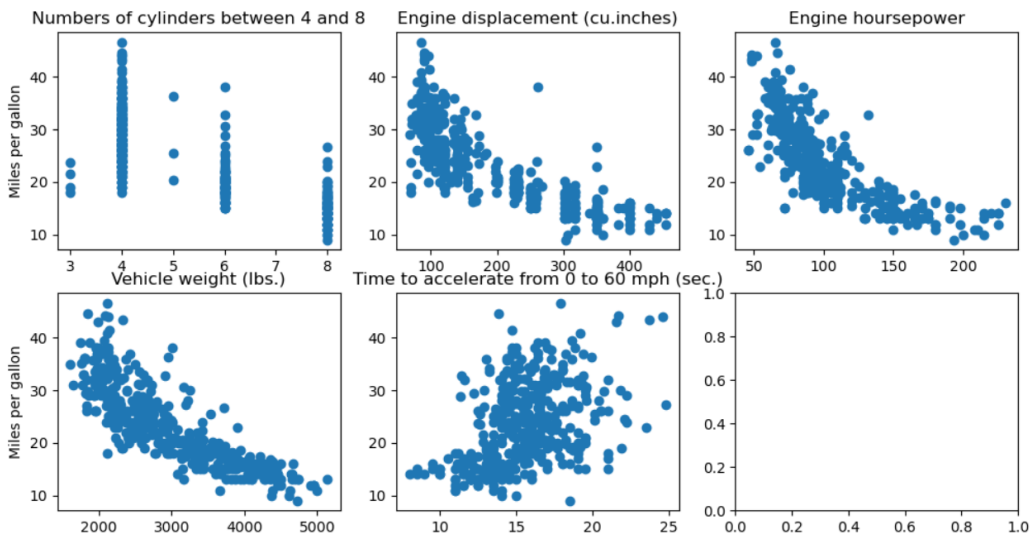
Model Correction

Auto Analytical Report

In this analysis, we are interested in reporting the fuel efficiency of the auto (which is measured by the recorded miles per gallon) based on a set of available independent variables. The Auto1.csv dataset contains the detailed information of 392 vehicles.

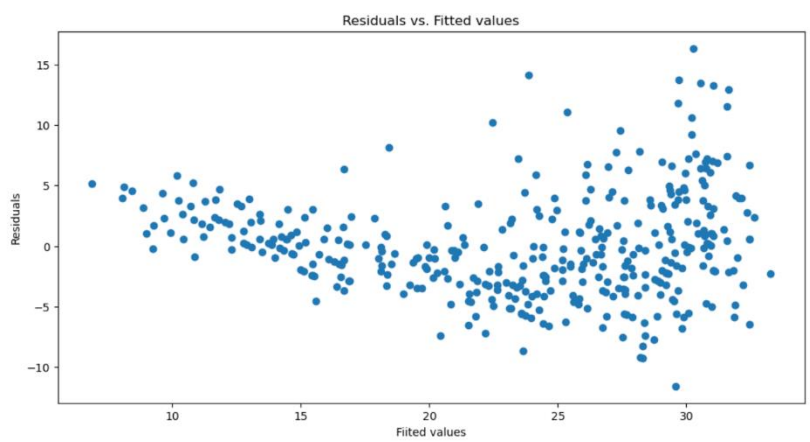
Initial diagnostic analysis revealed violations of linear regression assumptions, which includes linearity, zero mean, constant variance, and normality. Therefore, few of approaches are considered to correct these issues.

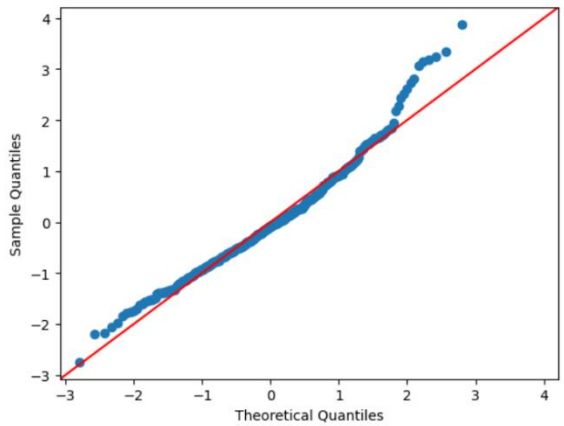
The reports below are the original OLS results and diagnostic checking.

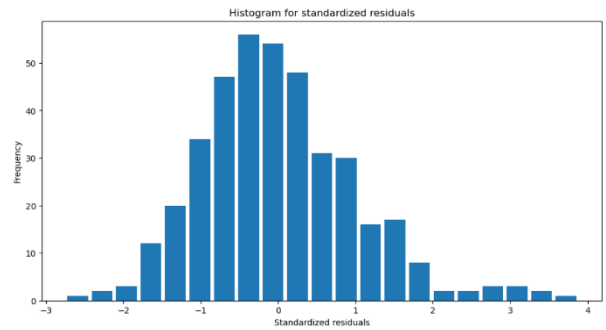



```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.708
Model:                            OLS   Adj. R-squared:                  0.704
Method:                 Least Squares   F-statistic:                     186.9
Date:                Thu, 29 Jan 2026   Prob (F-statistic):          9.82e-101
Time:                        15:40:08   Log-Likelihood:                -1120.1
No. Observations:                 392   AIC:                             2252.
Df Residuals:                     386   BIC:                             2276.
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const             46.2643      2.669     17.331      0.000      41.016      51.513
cylinders         -0.3979      0.411     -0.969      0.333      -1.205       0.409
displacement -8.313e-05      0.009     -0.009      0.993      -0.018       0.018
horsepower        -0.0453      0.017     -2.716      0.007      -0.078      -0.012
weight            -0.0052      0.001     -6.351      0.000      -0.007      -0.004
acceleration      -0.0291      0.126     -0.231      0.817      -0.276       0.218
==============================================================================
Omnibus:                       38.561   Durbin-Watson:                   0.865
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               52.737
Skew:                           0.706   Prob(JB):                     3.53e-12
Kurtosis:                       4.111   Cond. No.                     3.87e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.87e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```







- The plot pattern of residual plot is slightly U-shaped, indicating the potential of zero mean.
- The variance of residuals increases as the fitted response value increases, indicating the potential violation of constant variance.
- The points for Q-Q plot does not fall on a straight line. Instead, there is a heavy tail to the positive side, which indicates the violation of normality assumption.
- The histogram plot is slightly skewed to the right, indicating there is potential violation of the normality assumption.

1. To fix the violation of zero mean assumption, we first consider the polynomial regression. Specifically, please try adding a square term for those predictors which exhibit nonlinear relationship with the response (model 1). Report the fitted model, the R-square, and comment on the significance of the predictors. Based on this model, redo the diagnostic checking.

Step 1: Adding square term for those predictors which exhibit nonlinear relationship with the response

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
displacement = poly.fit_transform(data.iloc[:,2:3])
displacement = pd.DataFrame(displacement[:,1:], columns = ["displacement1", "displacement2"])
horsepower = poly.fit_transform(data.iloc[:,3:4])
horsepower = pd.DataFrame(horsepower[:,1:], columns = ["horsepower1", "horsepower2"])
weight = poly.fit_transform(data.iloc[:,4:5])
weight = pd.DataFrame(weight[:,1:], columns = ["weight1", "weight2"])
data = pd.concat([data.iloc[:,0:2],displacement, horsepower, weight, data.iloc[:,5:6]], axis = 1)
data
```

Step 2: Fit a multivariate linear regression model with all predictors using OLS function from statsmodels, and report the results.

```
import statsmodels.api as sm
ols = sm.OLS(data.iloc[:,0], sm.add_constant(data.iloc[:,1:9]))
lm = ols.fit()
print(lm.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.760
Model:                            OLS   Adj. R-squared:                  0.755
Method:                 Least Squares   F-statistic:                     151.8
Date:                Tue, 10 Feb 2026   Prob (F-statistic):          9.04e-114
Time:                        22:29:31   Log-Likelihood:                -1081.3
No. Observations:                 392   AIC:                             2181.
Df Residuals:                     383   BIC:                             2216.
Df Model:                           8
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const             62.5637      4.455     14.042      0.000      53.804      71.324
cylinders          0.7847      0.424      1.852      0.065      -0.048       1.618
displacement1     -0.0815      0.024     -3.356      0.001      -0.129      -0.034
displacement2      0.0001   4.35e-05      2.658      0.008    3.01e-05       0.000
horsepower1       -0.2969      0.053     -5.571      0.000      -0.402      -0.192
horsepower2        0.0007      0.000      3.984      0.000       0.000       0.001
weight1           -0.0021      0.003     -0.606      0.545      -0.009       0.005
weight2         9.793e-08   4.58e-07      0.214      0.831   -8.03e-07    9.99e-07
acceleration      -0.3866      0.133     -2.916      0.004      -0.647      -0.126
==============================================================================
Omnibus:                       39.429   Durbin-Watson:                   1.009
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               67.998
Skew:                           0.622   Prob(JB):                     1.72e-15
Kurtosis:                       4.618   Cond. No.                     2.52e+08
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.52e+08. This might indicate that there are
strong multicollinearity or other numerical problems.
```

As shown on the summery table above, the model 1 fitted linear regression is:

mpg = 62.5637 + 0.7847cylinders – 0.0815displacement1 + 0.0001displacement2 – 0.2969horsepower1 + 0.0007horsepower2 – 0.0021weight1 + 9.793* 10^{-8} weight2 – 0.3866acceleration

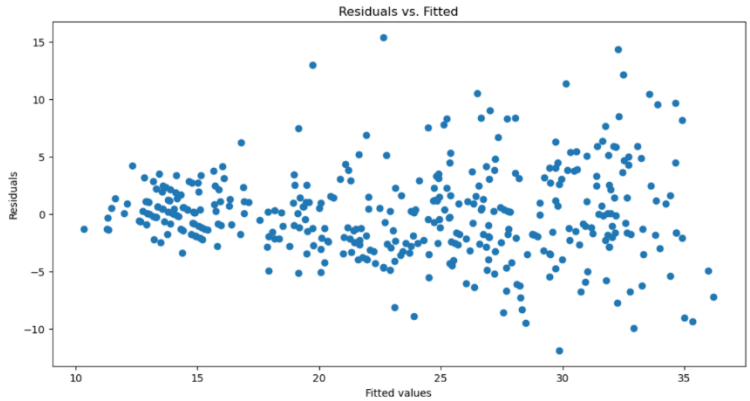
The R^2 of the model increases from 0.708 to 0.760, indicating that the model explains the variation slightly better.

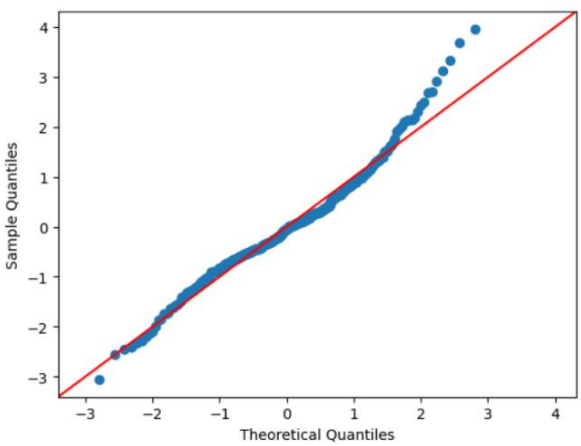
The squared term for displacement (displacement2) and horsepower (horsepower2) are significant, as both are less than 0.05, indicating that they are help to explain the miles per gallon response. Meanwhile, the cylinders, weight1 and weight2 are not significant in explaining the response value since their p-value are greater than 0.05.

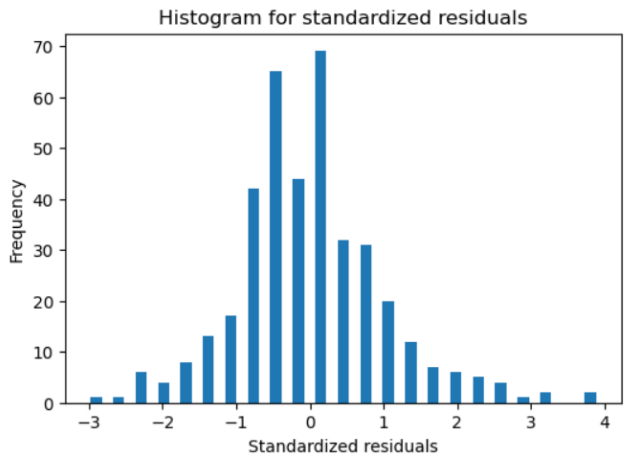
<u>Step 3: Redo the diagnostic test</u>

```
fitted_y = lm.fittedvalues
residual = lm.resid
plt.figure(figsize = (12,6))
plt.scatter(fitted_y,residual)
plt.title("Residuals vs. Fitted")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
```

```
from sklearn.preprocessing import scale
residual_norm = scale(residual)
sm.qqplot(residual_norm, line='45')
plt.figure(figsize = (6,4))
plt.title("Histogram for standardized residuals")
plt.hist(x=residual_norm, bins='auto', rwidth=0.5)
plt.xlabel("Standardized residuals")
plt.ylabel("Frequency")
```







- The residual plot seems to be improved and it mostly satisfies the zero mean assumption.
- As the fitted values increases, the variance of residuals slightly increases, indicating that there is a chance of violating the constant variance assumption.
- The Q-Q plot shows that the residuals has some values that are not close to the 45-degree reference line. Thus, the histogram show the right skewness. Therefore, both of Q-Q plot and the histogram suggests there is still violation in normality assumption, indicating the squared term transformation is not improving the data normality.

2. We can also try to fix the violation of zero mean assumption by considering the nonlinear transformation of those predictors which exhibit nonlinear relationship with the response (model 2). Specify the transformation that you are going to use, construct scatter plots of response variable versus each of the transformed predictors, and make the comments. Then report the new model, the R-square, and comment on the significance of the predictors. Based on this model, redo the diagnostic checking.

Step 1: Import the data again to set it as original data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
data = pd.read_csv("d:\\UAH GO CHARGERS\\Spring 26\\MSC450\\Module 4\\Assignment 3\\Auto-1.csv")
data.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	name
0	18.0	8	307.0	130	3504	12.0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	ford torino

Step 2: The scatter plots of mpg vs. displacement, mpg vs. horsepower, mpg vs. weight are showing the decrease nonlinear trend. Thus, applying log transformation into displacement, horsepower and weight can result in a more linear pattern. Therefore, log() pattern is used in model 2

[112]:
```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
displacement = np.log(data.iloc[:,2:3])
displacement = displacement.rename(columns={"displacement":"log_displacement"})
horsepower = np.log(data.iloc[:,3:4])
horsepower = horsepower.rename(columns={"horsepower":"log_horsepower"})
weight = np.log(data.iloc[:,4:5])
weight = weight.rename(columns={"weight":"log_weight"})
data = pd.concat([data.iloc[:,0:2], displacement, horsepower, weight, data.iloc[:,5:6]], axis = 1)
data
```

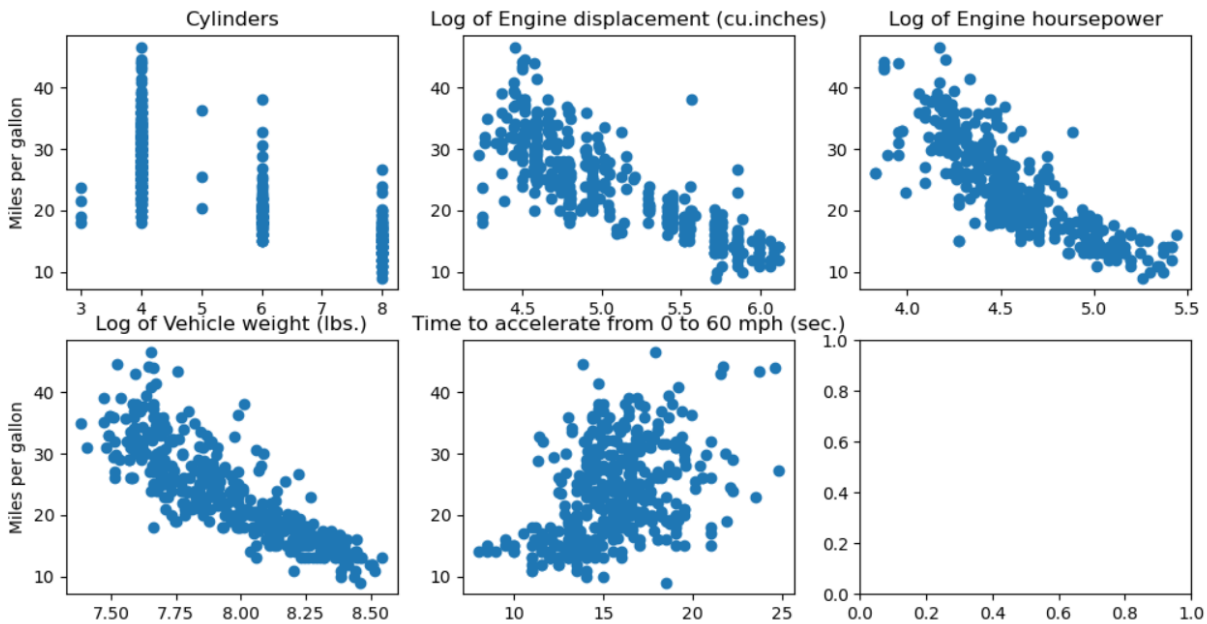
[112]:

	mpg	cylinders	log_displacement	log_horsepower	log_weight	acceleration
0	18.0	8	5.726848	4.867534	8.161660	12.0
1	15.0	8	5.857933	5.105945	8.214194	11.5
2	18.0	8	5.762051	5.010635	8.142063	11.0
3	16.0	8	5.717028	5.010635	8.141190	12.0
4	17.0	8	5.710427	4.941642	8.145840	10.5
...	...	...	...	...	...	...
387	27.0	4	4.941642	4.454347	7.933797	15.6
388	44.0	4	4.574711	3.951244	7.663877	24.6
389	32.0	4	4.905275	4.430817	7.738488	11.6
390	28.0	4	4.787492	4.369448	7.872836	18.6
391	31.0	4	4.779123	4.406719	7.908387	19.4

392 rows × 6 columns

Step 3: Construct scatter plots of response variable versus each of the transformed predictors

```
figure, axis = plt.subplots(2,3, figsize=(12,6))
axis[0,0].scatter(data.cylinders, data.mpg)
axis[0,0].set_title("Cylinders")
axis[0,0].set_ylabel("Miles per gallon")
axis[0,1].scatter(data.log_displacement, data.mpg)
axis[0,1].set_title("Log of Engine displacement (cu.inches)")
axis[0,2].scatter(data.log_horsepower, data.mpg)
axis[0,2].set_title("Log of Engine hoursepower")
axis[1,0].scatter(data.log_weight, data.mpg)
axis[1,0].set_title("Log of Vehicle weight (lbs.)")
axis[1,0].set_ylabel("Miles per gallon")
axis[1,1].scatter(data.acceleration, data.mpg)
axis[1,1].set_title("Time to accelerate from 0 to 60 mph (sec.)")
```



After the transformation, the plots of mpg versus log_displacement, log_horsepower, log_weight appear to have more linear relationship.

Step 4: Fit a multivariate linear regression model with all predictors using OLS function from statsmodels, and report the results.

```
import statsmodels.api as sm
ols = sm.OLS(data.iloc[:,0], sm.add_constant(data.iloc[:,1:6]))
lm = ols.fit()
print(lm.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                    mpg   R-squared:                       0.748
Model:                            OLS   Adj. R-squared:                  0.745
Method:                 Least Squares   F-statistic:                     229.3
Date:                Tue, 10 Feb 2026   Prob (F-statistic):          3.55e-113
Time:                        22:20:40   Log-Likelihood:                -1090.9
No. Observations:                 392   AIC:                             2194.
Df Residuals:                     386   BIC:                             2218.
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const            157.1480     12.362     12.712      0.000     132.843     181.453
cylinders          0.3237      0.355      0.912      0.362      -0.374       1.022
log_displacement  -4.2379      1.616     -2.623      0.009      -7.415      -1.061
log_horsepower   -11.2428      1.919     -5.858      0.000     -15.016      -7.469
log_weight        -7.0777      2.759     -2.565      0.011     -12.503      -1.652
acceleration      -0.3750      0.131     -2.868      0.004      -0.632      -0.118
==============================================================================
Omnibus:                       39.957   Durbin-Watson:                   0.974
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               62.427
Skew:                           0.667   Prob(JB):                     2.78e-14
Kurtosis:                       4.430   Cond. No.                     1.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

As shown in the OLS results, the model 2 fitted linear regression is:

mpg = 157.1480 + 0.3237cylinders – 4.2379log_displacement – 11.2428log_horsepower – 7.0777log_weight – 0.3750acceleration.

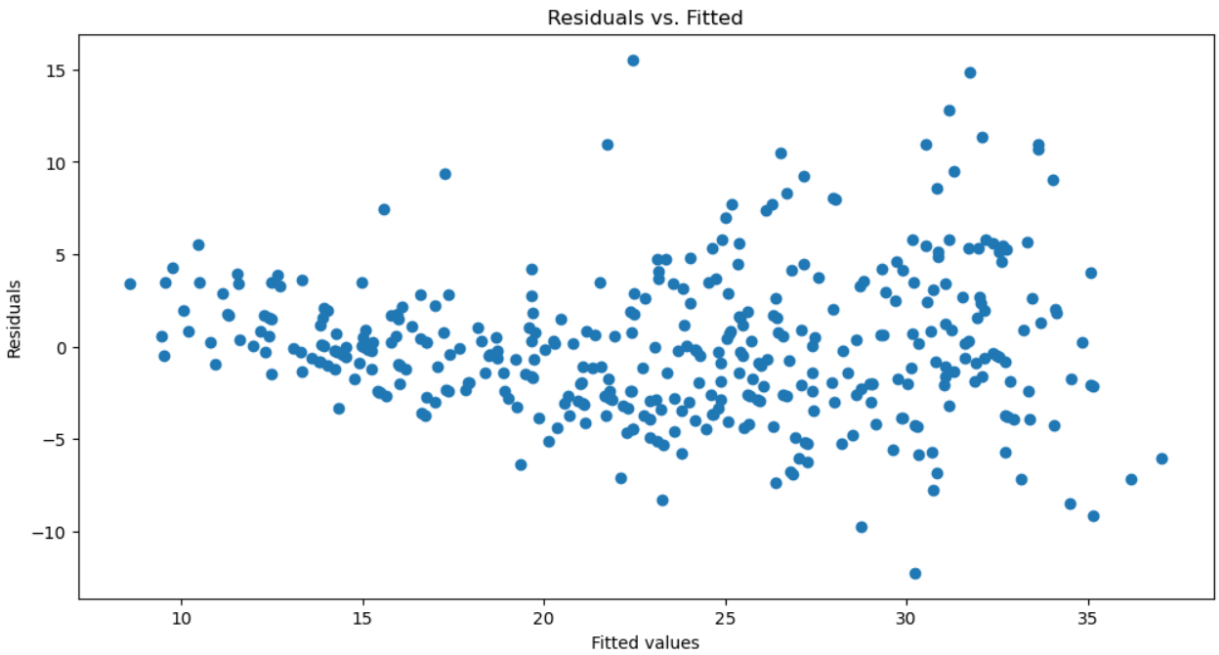
The R^2 result slightly increase from 0.708 to 0.748, indicating that the model regression explains the variation slightly better.

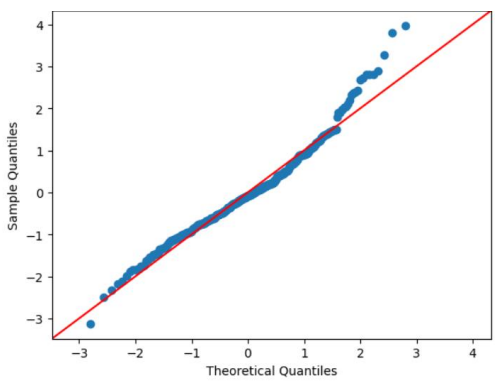
As we noticed, the log_displacement, log_horsepower, log_weight, and acceleration are all significant predictors in explaining the response value mpg since their p-values are less than 0.05. However, the cylinders data is not significant in the variance explanation of the regressions because its p-value is greater than 0.05.

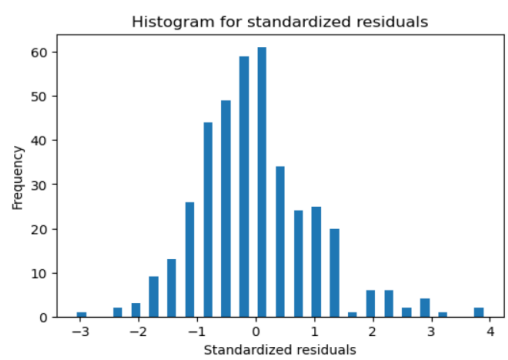
Step 5: Redo the diagnostic check

```
fitted_y = lm.fittedvalues
residual = lm.resid
plt.figure(figsize = (12,6))
plt.scatter(fitted_y,residual)
plt.title("Residuals vs. Fitted")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
```

```
from sklearn.preprocessing import scale
residual_norm = scale(residual)
sm.qqplot(residual_norm, line='45')
plt.figure(figsize = (6,4))
plt.title("Histogram for standardized residuals")
plt.hist(x=residual_norm, bins='auto', rwidth=0.5)
plt.xlabel("Standardized residuals")
plt.ylabel("Frequency")
```







- The residual plot seems to be improved and it mostly satisfies the zero mean assumption.
- As the fitted values increases, the variance of residuals slightly increases, indicating that there is a chance of violating the constant variance assumption.
- The Q-Q plot shows that the residuals has some values that are not close to the 45-degree reference line. Thus, the histogram show the right skewness. Therefore, both of Q-Q plot and the histogram suggests there is still violation in normality assumption, indicating the log transformation is not improving the data normality.

3. Next, we focus on the violation of constant variance and normality. According to the diagnostic check results that we reported in assignment 2, the variance of residuals increases as the fitted response values increases, and the residual distribution is right skewed. Thus, we first consider a log-transformation of the response variable and regress it on the original predictors (without being transformed) (model 3). Report the fitted model, the R-square, and comment on the significance of the predictors. Also, please interpret the coefficients for horsepower. Based on this model, redo the diagnostic checking.

Step 1: Import the data again to set it as original data

```
data = pd.read_csv("d:\\UAH GO CHARGERS\\Spring 26\\MSC450\\Module 4\\Assignment 3\\Auto-1.csv")
data.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	name
0	18.0	8	307.0	130	3504	12.0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	ford torino

Step 2: Use the log transformation on response variable

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
mpg = np.log(data.iloc[:,0:1])
mpg = mpg.rename(columns={"mpg":"log_mpg"})
data = pd.concat([mpg, data.iloc[:,1:6]], axis = 1)
data
```

	log_mpg	cylinders	displacement	horsepower	weight	acceleration
0	2.890372	8	307.0	130	3504	12.0
1	2.708050	8	350.0	165	3693	11.5
2	2.890372	8	318.0	150	3436	11.0
3	2.772589	8	304.0	150	3433	12.0
4	2.833213	8	302.0	140	3449	10.5
...	...	...	...	...	...	...
387	3.295837	4	140.0	86	2790	15.6
388	3.784190	4	97.0	52	2130	24.6
389	3.465736	4	135.0	84	2295	11.6
390	3.332205	4	120.0	79	2625	18.6
391	3.433987	4	119.0	82	2720	19.4

Step 3: Fit a multivariate linear regression model with all predictors using OLS function from statsmodels, and report the results.

```
import statsmodels.api as sm
ols = sm.OLS(data.iloc[:,0], sm.add_constant(data.iloc[:,1:6]))
lm = ols.fit()
print(lm.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                log_mpg   R-squared:                       0.791
Model:                            OLS   Adj. R-squared:                  0.789
Method:                 Least Squares   F-statistic:                     292.9
Date:                Tue, 10 Feb 2026   Prob (F-statistic):          6.18e-129
Time:                        22:53:30   Log-Likelihood:                 174.31
No. Observations:                 392   AIC:                            -336.6
Df Residuals:                     386   BIC:                            -312.8
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const            4.2203      0.098     42.956      0.000       4.027       4.413
cylinders       -0.0260      0.015     -1.718      0.087      -0.056       0.004
displacement -1.803e-05      0.000     -0.054      0.957      -0.001       0.001
horsepower      -0.0028      0.001     -4.513      0.000      -0.004      -0.002
weight          -0.0002   3.01e-05     -6.687      0.000      -0.000      -0.000
acceleration    -0.0057      0.005     -1.235      0.218      -0.015       0.003
==============================================================================
Omnibus:                        5.326   Durbin-Watson:                   0.890
Prob(Omnibus):                  0.070   Jarque-Bera (JB):                5.251
Skew:                           0.223   Prob(JB):                       0.0724
Kurtosis:                       3.351   Cond. No.                     3.87e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.87e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

As the OLS results shown above, the model 3 fitted regression is:

log_mpg = 4.2203 – 0.026cylinders – 1.803 * 10^{-5} displacement – 0.0028horsepower – 0.0002weight – 0.0057acceleration

The R^2 result slightly increase from 0.708 to 0.791, indicating that the model regression explains the variation slightly better.

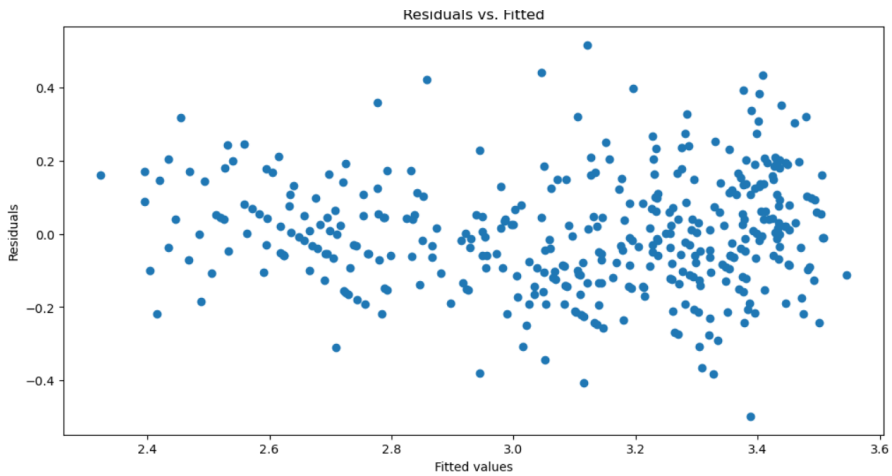
Based on the results, the horsepower and weight predictors are all significant in explaining the response value mpg since their p-values are less than 0.05. However, the cylinders, displacement, and acceleration predictors are not significant in the variance explanation of the regressions because their p-value are greater than 0.05.

Step 4: Interpret the coefficients of the horsepower

Every increase unit in horsepower is associated with approximately a 0.28% decrease in mpg.

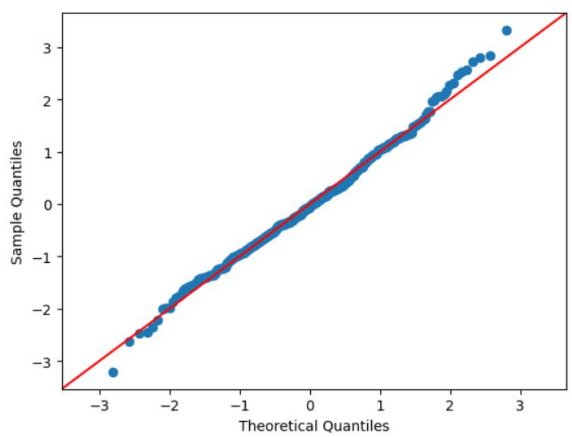
Step 5: Redo the diagnostic check

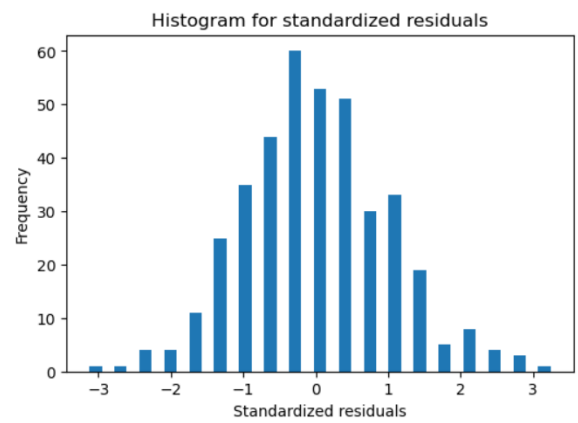
```
fitted_y = lm.fittedvalues
residual = lm.resid
plt.figure(figsize = (12,6))
plt.scatter(fitted_y,residual)
plt.title("Residuals vs. Fitted")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
```



- In residual plot, the zero mean assumption is mostly satisfied since the data scatters slightly even around 0. Thus, the constant variance assumption seems to be satisfied as fitted values increase, the variance of residuals stays constant.

```
from sklearn.preprocessing import scale
residual_norm = scale(residual)
sm.qqplot(residual_norm, line='45')
plt.figure(figsize = (6,4))
plt.title("Histogram for standardized residuals")
plt.hist(x=residual_norm, bins='auto', rwidth=0.5)
plt.xlabel("Standardized residuals")
plt.ylabel("Frequency")
```





- The Q-Q plot shows that the residuals has most of the values close to the 45-degree reference line. Thus, the histogram shows the symmetric compared to the previous models. Therefore, both of Q-Q plot and the histogram suggests there is still slight deviation at the tails, the distribution appears approximately normal, indicating the log of response value improves the normality assumption.

4. Let's now regress the log-transformed response variable on the transformed predictors that we used in model 2 (model 4). Report the fitted model, the R-square, and comment on the significance of the predictors. Based on this model, redo the diagnostic checking.

Step 1: Import the data again to set it as original data

```
[264]: data = pd.read_csv("d:\\UAH GO CHARGERS\\Spring 26\\MSC450\\Module 4\\Assignment 3\\Auto-1.csv")
       data.head()
```

[264]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	name
0	18.0	8	307.0	130	3504	12.0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	ford torino

Step 2: Use the log transformation on response variable on the transformed predictors that we used in model 2.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
mpg = np.log(data.iloc[:,0:1])
mpg = mpg.rename(columns={"mpg":"log_mpg"})
displacement = np.log(data.iloc[:,2:3])
displacement = displacement.rename(columns={"displacement":"log_displacement"})
horsepower = np.log(data.iloc[:,3:4])
horsepower = horsepower.rename(columns={"horsepower":"log_horsepower"})
weight = np.log(data.iloc[:,4:5])
weight = weight.rename(columns={"weight":"log_weight"})
data = pd.concat([mpg,data.iloc[:,1:2],displacement, horsepower, weight, data.iloc[:,5:6]], axis = 1)
data
```

	log_mpg	cylinders	log_displacement	log_horsepower	log_weight	acceleration
0	2.890372	8	5.726848	4.867534	8.161660	12.0
1	2.708050	8	5.857933	5.105945	8.214194	11.5
2	2.890372	8	5.762051	5.010635	8.142063	11.0
3	2.772589	8	5.717028	5.010635	8.141190	12.0
4	2.833213	8	5.710427	4.941642	8.145840	10.5
...	...	...	...	...	...	...
387	3.295837	4	4.941642	4.454347	7.933797	15.6
388	3.784190	4	4.574711	3.951244	7.663877	24.6
389	3.465736	4	4.905275	4.430817	7.738488	11.6
390	3.332205	4	4.787492	4.369448	7.872836	18.6
391	3.433987	4	4.779123	4.406719	7.908387	19.4

Step 3: Fit a multivariate linear regression model with all predictors using OLS function from statsmodels, and report the results.

```
import statsmodels.api as sm
ols = sm.OLS(data.iloc[:,0], sm.add_constant(data.iloc[:,1:6]))
lm = ols.fit()
print(lm.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                log_mpg   R-squared:                       0.805
Model:                            OLS   Adj. R-squared:                  0.803
Method:                 Least Squares   F-statistic:                     319.0
Date:                Tue, 10 Feb 2026   Prob (F-statistic):          1.20e-134
Time:                        23:29:39   Log-Likelihood:                 187.69
No. Observations:                 392   AIC:                            -363.4
Df Residuals:                     386   BIC:                            -339.6
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const              8.7657      0.474     18.505      0.000       7.834       9.697
cylinders         -0.0206      0.014     -1.515      0.131      -0.047       0.006
log_displacement  -0.0901      0.062     -1.455      0.146      -0.212       0.032
log_horsepower    -0.4844      0.074     -6.588      0.000      -0.629      -0.340
log_weight        -0.3303      0.106     -3.123      0.002      -0.538      -0.122
acceleration      -0.0155      0.005     -3.100      0.002      -0.025      -0.006
==============================================================================
Omnibus:                        9.808   Durbin-Watson:                   0.989
Prob(Omnibus):                  0.007   Jarque-Bera (JB):               12.300
Skew:                           0.245   Prob(JB):                      0.00213
Kurtosis:                       3.717   Cond. No.                     1.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

As the OLS results shown above, the model 4 fitted regression is:

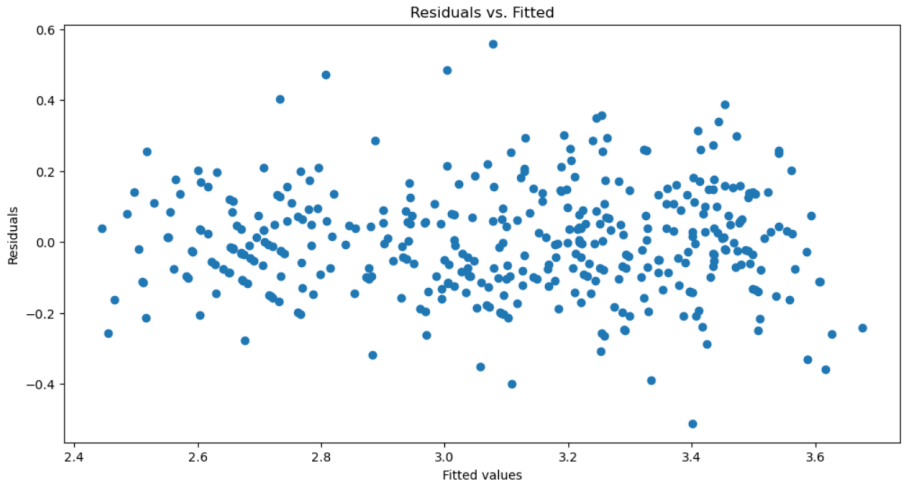
log_mpg = 8.7657 – 0.0206cylinders – 0.0901log_displacement – 0.4844log_horsepower – 0.3303log_weight – 0.0155acceleration

The R^2 result significantly increase from 0.708 to 0.805, indicating that the model regression explains the variation better.

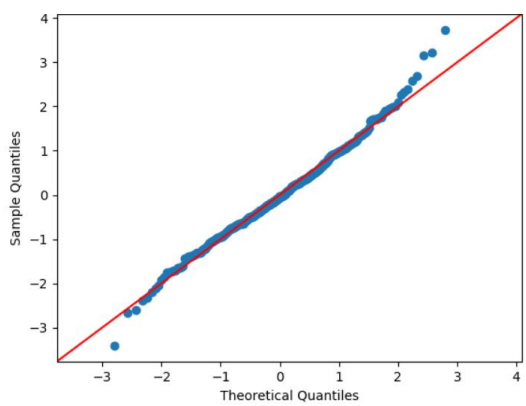
Based on the results, the log_hoursepower, log_weight and acceleration predictors are all significant in explaining the response value mpg since their p-values are less than 0.05. However, the cylinders and log_displacement predictors are not significant in the variance explanation of the regressions because their p-value are greater than 0.05.

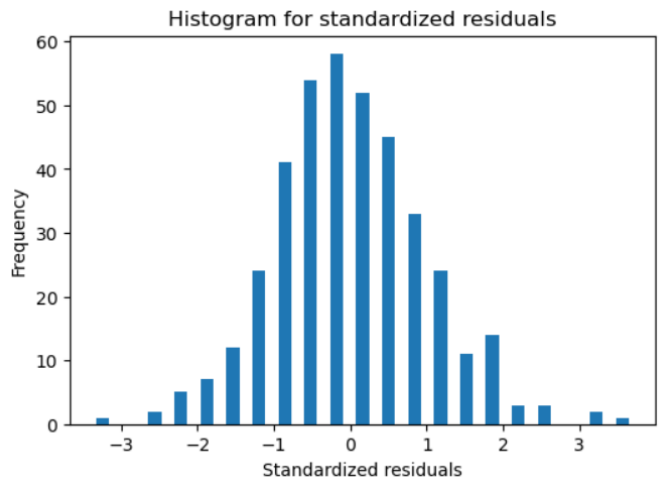
Step 4: Redo the diagnostic check

```
fitted_y = lm.fittedvalues
residual = lm.resid
plt.figure(figsize = (12,6))
plt.scatter(fitted_y,residual)
plt.title("Residuals vs. Fitted")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
```



- In residual plot, the zero mean assumption is mostly satisfied since the data scatters randomly with even values for each side of 0. Thus, the constant variance assumption seems to be satisfied as fitted values increase, the variance of residuals stays constant.





- The Q-Q plot shows that the residuals has most of the values close to the 45-degree reference line. Thus, the histogram shows the symmetric compared to the previous models. Therefore, both of Q-Q plot and the histogram suggests there is still slight deviation at the tails, the distribution appears approximately normal.

5. (Bonus question) Based on model 4, please interpret the coefficients for horsepower.

For every percentage increase of the horsepower, it will cause to 0.4844% decrease in mpg.

6. Finally, let's try the Box-Cox transformation. Please transform the response variable using the optimal lambda value with the highest log-likelihood and regress it on the transformed predictors that we used in model 2 & 4 (model 5). Report the fitted model, the R-square, and comment on the significance of the predictors. Based on this model, redo the diagnostic checking.

Step 1: Import the data again to set it as original data

```
#model 5
```

```
data = pd.read_csv("d:\\UAH GO CHARGERS\\Spring 26\\MSC450\\Module 4\\Assignment 3\\Auto-1.csv")
data.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	name
0	18.0	8	307.0	130	3504	12.0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	ford torino

Step 2: Use log transformation for same predictors in model 2

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
displacement = np.log(data.iloc[:,2:3])
displacement = displacement.rename(columns={"displacement":"log_displacement"})
horsepower = np.log(data.iloc[:,3:4])
horsepower = horsepower.rename(columns={"horsepower":"log_horsepower"})
weight = np.log(data.iloc[:,4:5])
weight = weight.rename(columns={"weight":"log_weight"})
data = pd.concat([data.iloc[:,0:2],displacement, horsepower, weight, data.iloc[:,5:6]], axis = 1)
data
```

	mpg	cylinders	log_displacement	log_horsepower	log_weight	acceleration
0	18.0	8	5.726848	4.867534	8.161660	12.0
1	15.0	8	5.857933	5.105945	8.214194	11.5
2	18.0	8	5.762051	5.010635	8.142063	11.0
3	16.0	8	5.717028	5.010635	8.141190	12.0
4	17.0	8	5.710427	4.941642	8.145840	10.5
...	...	...	...	...	...	...
387	27.0	4	4.941642	4.454347	7.933797	15.6
388	44.0	4	4.574711	3.951244	7.663877	24.6
389	32.0	4	4.905275	4.430817	7.738488	11.6
390	28.0	4	4.787492	4.369448	7.872836	18.6
391	31.0	4	4.779123	4.406719	7.908387	19.4

392 rows × 6 columns

Step 3: Using Box-Cox to determine log-likelihood of the response value

```
import scipy.stats as stats
Z, fit_lambda = stats.boxcox(data.mpg)
print(fit_lambda)
```

```
0.19123084478293514
```

Step 4: Fit a multivariate linear regression model with all predictors using OLS function from statsmodels, and report the results.

```
import statsmodels.api as sm
ols = sm.OLS(Z, sm.add_constant(data.iloc[:,1:6]))
lm = ols.fit()
print(lm.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.798
Model:                            OLS   Adj. R-squared:                  0.795
Method:                 Least Squares   F-statistic:                     305.0
Date:                Wed, 11 Feb 2026   Prob (F-statistic):          1.26e-131
Time:                        00:05:47   Log-Likelihood:                -50.812
No. Observations:                 392   AIC:                             113.6
Df Residuals:                     386   BIC:                             137.5
Df Model:                           5
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const             14.5514      0.870     16.718      0.000      12.840      16.263
cylinders         -0.0254      0.025     -1.017      0.310      -0.075       0.024
log_displacement  -0.1980      0.114     -1.740      0.083      -0.422       0.026
log_horsepower    -0.8730      0.135     -6.461      0.000      -1.139      -0.607
log_weight        -0.5911      0.194     -3.042      0.003      -0.973      -0.209
acceleration      -0.0283      0.009     -3.071      0.002      -0.046      -0.010
==============================================================================
Omnibus:                       13.044   Durbin-Watson:                   0.979
Prob(Omnibus):                  0.001   Jarque-Bera (JB):               16.523
Skew:                           0.313   Prob(JB):                     0.000258
Kurtosis:                       3.787   Cond. No.                     1.25e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.25e+03. This might indicate that there are
strong multicollinearity or other numerical problems.
```

As the OLS results shown above, the model 5 fitted regression is:

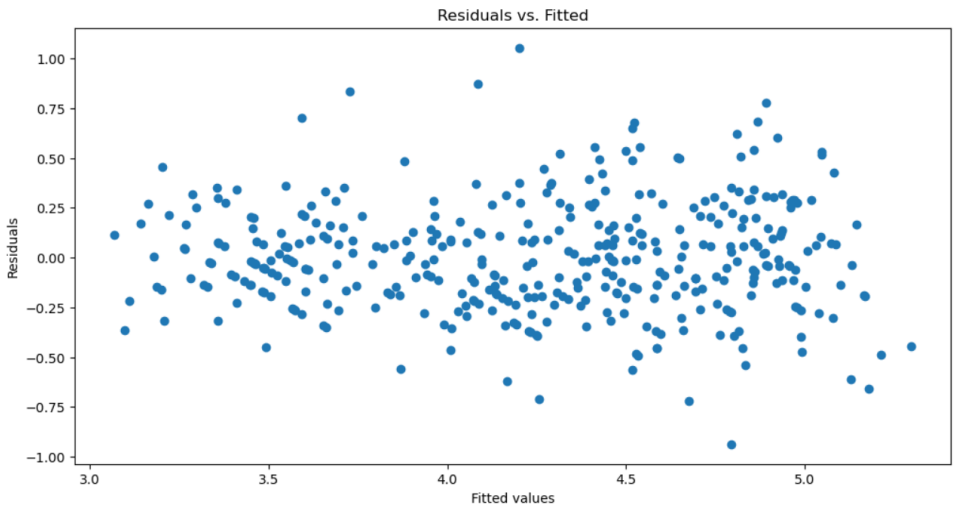
Z (lambda mpg) = 14.5514 – 0.0254cylinders – 0.1980log_displacement – 0.8730log_horsepower – 0.5911log_weight – 0.0283acceleration

The R^2 result slightly increase from 0.708 to 0.798, indicating that the model regression explains the variation better.

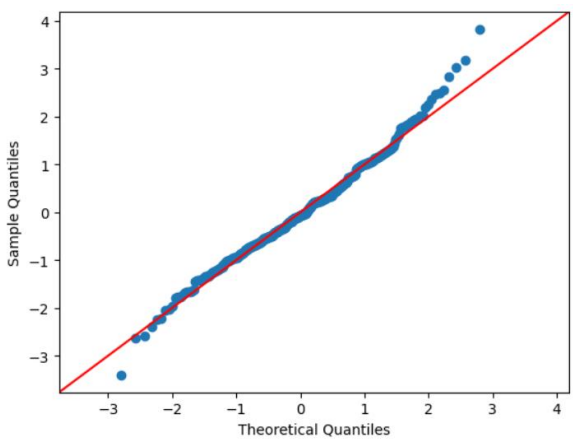
Based on the results, the log_hoursepower, log_weight and acceleration predictors are all significant in explaining the response value mpg since their p-values are less than 0.05. However, the cylinders

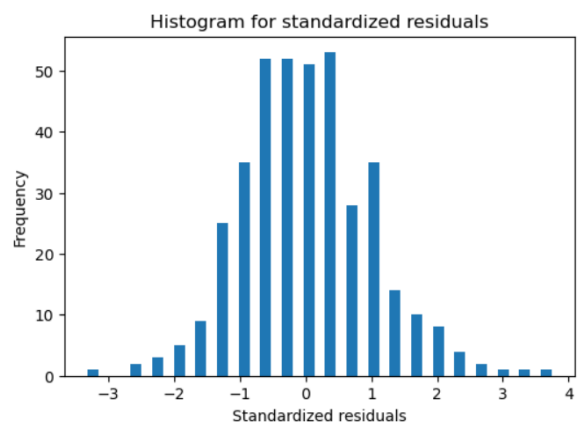
and log_displacement predictors are not significant in the variance explanation of the regressions because their p-value are greater than 0.05.

Step 5: Redo the diagnostic check



- In residual plot, the zero mean assumption is mostly satisfied since the data scatters randomly with even values for each side of 0. Thus, the constant variance assumption seems to be satisfied as fitted values increase, the variance of residuals stays constant.





- The Q-Q plot shows that the residuals has some of the values are not close to the 45-degree reference line. Thus, the histogram shows the slightly right skewness compared to the previous models. Therefore, both of Q-Q plot and the histogram suggests there is still slight deviation at the tails, indicating a small violation of normality assumption after the Box-Cox transformation.

7. Please compare all the models that we have just constructed (model 1 – 5). Which is best model that you would like to report, and briefly explain why.

The following table is the comparison between all the models (model 1-5):

	Model 1	**Model 2**	**Model 3**	**Model 4**	**Model 5**
R^2	0.760	0.748	0.791	0.805	0.798
Significant Predictors	Displacement1,2 Horsepower1,2 Acceleration	log_dsplacement log_horsepower log_weight Acceleration	Horsepower Weight	log_horsepower log_weight Acceleration	log_horsepower log_weight Acceleration
Zero Mean	Slightly violated	Slightly violated	Mostly satisfied	Mostly satisfied	Mostly satisfied
Constant Variance	Slightly violated	Slightly violated	Mostly satisfied	Mostly satisfied	Mostly satisfied
Normality	Violated (Right skewness)	Violated (Right skewness)	Approximately normal	Approximately normal	Slight violated (Right skewness)

The most model I would like to report is Model 4 as the following reasons:

First, Model 4 has the highest $R^2 = 0.805$, indicating that its regression model explains the most variance of response data (mpg) comparing to other models.

Second, Model 4 reasonably satisfies the assumptions of the linear regression. The residuals plot show that the residuals randomly scatter around zero, suggesting the zero mean assumption is satisfied. Moreover, as the fitted value increases, the variance of the residuals is approximately constant. Thus, the Q-Q plot and histogram both suggests the residuals are approximately normally distributed.

Third, the predictors of the model are statistically significant and interpretable.

Last, the Box-Cox transformation results $\lambda = 0.19$ in Model 5, which is close to 0, suggesting the log transformation is appropriate.

Therefore, Model 4 provides the best combination between assumption satisfaction, explanation of variability, and interpretability.